

Họ và tên:

Mã số Sinh viên: 24520346

Lớp:

HỆ ĐIỀU HÀNH BÁO CÁO LAB 3

TÓM TẮT BÁO CÁO

1. CLASSWORK

	EX 1	EX 2	EX 3	EX 4
Phương pháp	☐	☒	☒	☒
Minh chứng	☒	☒	☒	☒
Giải thích	☒	☒	☒	☒

2. HOMEWORK

	A
Method	☒
Proof	☒
Explanation	☒

Tự đánh giá:

**Lưu ý: Xuất file PDF, đặt tên theo cú pháp:*

<MSSV>_LABx.pdf

REPORT

3.5. Bài tập tại lớp

3.5.1. Thực hiện Ví dụ 3-1, Ví dụ 3-2, Ví dụ 3-3, Ví dụ 3-4 giải thích code và kết quả nhận được?

- Ví dụ 3-1:

```

32 # IT007 Operating System
31 # Phạm Ngọc Dũng, 24520346
30 # File: test_fork.c
29 #####
28 #include <stdio.h>
27 #include <stdlib.h>
26 #include <unistd.h>
25 #include <sys/wait.h>
24 #include <sys/types.h>
23
22 int
21 main(int argc, char *argv[])
20 {
19     pid_t pid;
18     pid = fork();
17
16     if (pid > 0) {
15         printf("PARENTS | PID = %ld | PPID = %ld\n",
14             (long)getpid(), (long)getppid());
13         if (argc > 2)
12             printf("PARENTS | There are %d arguments\n", argc - 1);
11         wait(NULL);
10     }
9
8     if (pid == 0) {
7         printf("CHILDREN | PID = %ld | PPID = %ld\n",
6             (long)getpid(), (long)getppid());
5         printf("CHILDREN | List of arguments: \n");
4         for (int i = 1; i < argc; i++) {
3             printf("%s\n", argv[i]);
2         }
1     }
35
1     exit(0);
2 }
test_fork.c 99% « 35, 1
[0] 0:vi* "tch@localIPC vd" 13:49 05-Nov-25

```

Giải thích: chương trình dùng fork() để tạo tiến trình con rồi cho in ID của tiến trình (PID) và tiến trình cha (PPID). Trên PARENTS, PPID là PID của shell được dùng để chạy chương trình, trên CHILDREN, PID riêng, khác PARENTS, PPID là PID của PARENTS. Phần PARENTS in số tham số nhận vào khi có từ 2 tham số trở lên. Phần CHILDREN in các tham số nhận vào.

- Ví dụ 3-2:

```
11 #include <stdio.h>
10 #include <stdlib.h>
9 #include <unistd.h>
8 #include <sys/wait.h>
7 #include <sys/types.h>
6
5 int
4 main(int argc, char *argv[])
3 {
2 >     pid_t pid;
1 >     pid = fork();
18 |
1 >     if (pid > 0)
2 >     {
3 >         printf("PARENTS | PID = %d | PPID = %d\n",
4 >             (long)getpid(), (long)getppid());
5 >         if (argc > 2)
6 >             printf("PARENTS | There are %d arguments\n", argc - 1);
7 >         wait(NULL);
8 >     }
9
10 >     if (pid == 0)
11 >     {
12 >         execl("./count.sh", "./count.sh", "10", NULL);
13 >         printf("CHILDREN | PID = %d | PPID = %d\n",
14 >             (long)getpid(), (long)getppid());
15 >         printf("CHILDREN | List of arguments: \n");
16 >         for (int i = 1; i < argc; i++)
17 >         {
18 >             printf("%s\n", argv[i]);
19 >         }
20 >     }
21
22 >     exit(0);
23 }
```

```
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc test_execl.c -o test_execl
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./test_execl ThamSo1 ThamSo2 ThamSo3
PARENTS | PID = 16658 | PPID = 26706
PARENTS | There are 3 arguments
Implementing: ./count.sh
PPID of count.sh:
tch 15572 15244 0 13:53 pts/4 00:00:00 vi count.sh
tch 16659 16658 0 13:53 pts/3 00:00:00 /bin/bash ./count.sh 10
tch 16661 16659 0 13:53 pts/3 00:00:00 grep count.sh
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$

1 #!/bin/bash
2
1 echo "Implementing: $0"
2 echo "PPID of count.sh: "
3 ps -ef | grep count.sh
4
5 i=1
6
7 while [ $i -le $1 ]
8 do
9     echo $i >> count.txt
10    i=$((i + 1))
11    sleep 1
12 done
13 exit 0
~
~
~
~
```

test_execl.c 44% « 18, 1 count.sh 7% « 2, 1

[0] 0:bash+ "tch@localPC vd" 13:54 05-Nov-25

Giải thích: chương trình thực thi script count.sh với tham số 10 ở tiến trình con. Trong script, lệnh `ps -ef | grep count.sh` liệt kê một số thông tin của tất cả các lệnh đang chạy có chứa count.sh, sau đó script ghi các số từ 1 tới 10 vào tệp count.txt, mỗi lần ghi tạm ngưng 1 giây. Phần chương trình còn lại tương tự ví dụ 3-1. Do execl đề lên tiến trình gọi nó nên các dòng printf bị ẩn khỏi màn hình.

- Ví dụ 3-3:

```

11 ./#####
10 # University of Information Technology
9 # IT007 Operating System
8 # Phạm Ngọc Dũng, 24520346
7 # File: test_system.c
6 #####/
5 #include <stdio.h>
4 #include <stdlib.h>
3 #include <unistd.h>
2 #include <sys/wait.h>
1 #include <sys/types.h>
12
1 int
2 main(int argc, char *argv[])
3 {
4     printf("PARENTS | PID = %ld | PPID = %ld\n",
5           (long)getpid(), (long)getppid());
6
7     system("./count.sh 10");
8
9     printf("PARENTS | List of arguments: \n");
10    for (int i = 1; i < argc; i++)
11    {
12        printf("%s\n", argv[i]);
13    }
14
15    exit(0);
16 }
~
~
~
~
~
~
test_system.c          54% « 12, 1
[0] 0:bash*

```

```

tch::~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./test_system ThamSo1 ThamSo2 ThamSo3
PARENTS | PID = 18783 | PPID = 26706
Implementing: ./count.sh
PPID of count.sh:
tch    15572 15244  0 13:53 pts/4    00:00:00 vi count.sh
tch    18784 18783  0 13:55 pts/3    00:00:00 /bin/bash ./count.sh 10
tch    18786 18784  0 13:55 pts/3    00:00:00 grep count.sh
PARENTS | List of arguments:
ThamSo1
ThamSo2
ThamSo3
tch::~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$

```

```

1 #!/bin/bash
2
1 echo "Implementing: $0"
2 echo "PPID of count.sh: "
3 ps -ef | grep count.sh
4
5 i=1
6
7 while [ $i -le $1 ]
8 do
9     echo $i >> count.txt
10    i=$((i + 1))
11    sleep 1
12 done
13 exit 0
~
~
~
~
~
~
count.sh              7% « 2, 1
"tch@localPC vd" 13:55 05-Nov-25

```

Giải thích: chương trình in PID và PPID, thực thi `count.sh` với tham số 10 và in ra các tham số nhập vào nếu số tham số lớn hơn 2. Khi sử dụng `system()`, tiến trình cha chờ tiến trình con thực thi xong rồi mới thực thi tiếp nên các dòng `printf()` bên dưới vẫn có thể in kết quả ra màn hình. Script `count.sh` hoạt động giống ví dụ trước.

BÁO CÁO THỰC HÀNH HỆ ĐIỀU HÀNH - GVHD: Thân Thế Tùng

- Ví dụ 3-4:

```
34 #include <sys/shm.h>
33 #include <sys/stat.h>
32 #include <unistd.h>
31 #include <sys/mman.h>
30
29 int main()
28 {
27     /* the size (in bytes) of shared memory object */
26     const int SIZE = 4096;
25     /* name of the shared memory object */
24     const char *name = "OS";
23     /* shared memory file descriptor */
22     int fd;
21     /* pointer to shared memory object */
20     char *ptr;
19     /* create the shared memory object */
18     fd = shm_open(name, O_CREAT | O_RDWR, 0660);
17     /* configure the size of the shared memory object */
16     ftruncate(fd, SIZE);
15     /* memory map the shared memory object */
14     ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
13     /* write to the shared memory object */
12     strcpy(ptr, "Hello Process B");
11     /* wait until Process B updates the shared memory segment */
10     while (strcmp(ptr, "Hello Process B", 15) == 0)
9     {
8         printf("Waiting Process B update shared memory\n");
7         sleep(1);
6     }
5     printf("Memory updated: %s\n", (char *)ptr);
4     /* unmap the shared memory segment and close the file descriptor */
3     munmap(ptr, SIZE);
2     close(fd);
1     return 0;
47 }
```

```
34 #include <string.h>
33 #include <fcntl.h>
32 #include <sys/shm.h>
31 #include <sys/stat.h>
30 #include <unistd.h>
29 #include <sys/mman.h>
28
27 int main()
26 {
25     /* the size (in bytes) of shared memory object */
24     const int SIZE = 4096;
23     /* name of the shared memory object */
22     const char *name = "OS";
21     /* shared memory file descriptor */
20     int fd;
19     /* pointer to shared memory object */
18     char *ptr;
17     /* create the shared memory object */
16     fd = shm_open(name, O_RDWR, 0660);
15     /* memory map the shared memory object */
14     ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
13     /* read from the shared memory object */
12     printf("Read shared memory: ");
11     printf("%s\n", (char *)ptr);
10     /* update the shared memory object */
9     strcpy(ptr, "Hello Process A");
8     printf("Shared memory updated: %s\n", ptr);
7     sleep(5);
6     /* unmap the shared memory segment and close the file descriptor */
5     munmap(ptr, SIZE);
4     // remove the shared memory segment
3     shm_unlink(name);
2     close(fd);
1     return 0;
46 }
```

test_shm_A.c 100% « 47, 1 test_shm_B.c 100% « 45, 1

[0] 0:vi+ "tch@localPC vd" 02:56 06-Nov-25

```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc test_shm_A.c -o test_shm_A
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./test_shm_A
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Waiting Process B update shared memory
Memory updated: Hello Process A
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$

tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc test_shm_B.c -o test_shm_B
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./test_shm_B
Read shared memory: Hello Process B
Shared memory updated: Hello Process A
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

[0] 0:vim+ "tch@localPC vd" 02:55 06-Nov-25

Giải thích: chương trình A tạo vùng nhớ chia sẻ có tên OS rồi truyền vào chuỗi "Hello Process B", sau đó chờ chương trình B. Chương trình B đọc vùng nhớ chia sẻ, in chuỗi trong vùng nhớ rồi truyền vào chuỗi mới "Hello

Process A". Chương trình A đợi tới khi chuỗi ban đầu thay đổi rồi in chuỗi mới rồi thoát, chương trình B hủy vùng nhớ chia sẻ rồi thoát.

3.5.2. Viết chương trình time.c thực hiện đo thời gian thực thi của một lệnh shell. Chương trình sẽ được chạy với cú pháp `./time <command>` với `<command>` là lệnh shell muốn đo thời gian thực thi.

- **Phương pháp:** lấy thời gian trước rồi thực thi lệnh trong tiến trình con, tiến trình cha đợi rồi lấy thời gian sau, trừ thời gian sau cho thời gian trước ta được thời gian thực thi.

```
7 #include <sys/mman.h>
6 #include <sys/time.h>
5 #include <sys/wait.h>
4 #include <err.h>
3 #include <stdio.h>
2 #include <stdlib.h>
1 #include <unistd.h>
8
1 int
2 main(int argc, char *argv[])
3 {
4     struct timeval *first, second;
5
6     if (argc < 2)
7     {
8         errx(0, "Cách dùng: %s <trình> [tham số ...]", argv[0]);
9     }
10    first = mmap(0, sizeof(struct timeval), PROT_READ | PROT_WRITE,
11                MAP_SHARED | MAP_ANON, -1, 0);
12    if (first == MAP_FAILED)
13    {
14        err(1, "mmap");
15    }
16    switch (fork()) {
17    case -1:
18        err(2, "fork");
19    case 0:
20        gettimeofday(first, NULL);
21        execvp(argv[1], &argv[1]);
22        err(3, "%s", argv[1]);
23    default:
24        wait(NULL);
25        gettimeofday(&second, NULL);
26        timersub(&second, first, first);
27        printf("Thời gian thực thi: %ld.%06ld giây.\n",
28              first->tv_sec, first->tv_usec);
29    }
30    munmap(first, sizeof(struct timeval));
31    return 0;
32 }
```

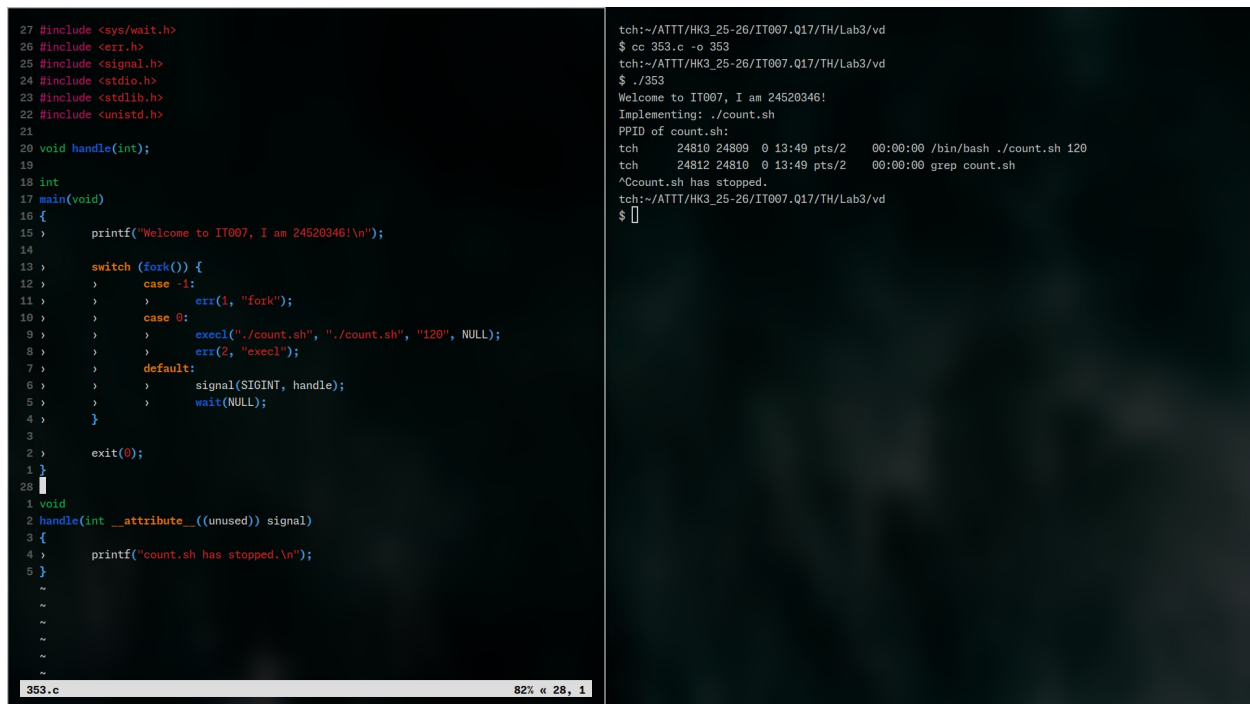
```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc time.c -o time
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time ls -lh
total 106K
-rwxr-xr-x 1 tch users 5.6K Nov 6 13:49 353
-rw-r--r-- 1 tch users 491 Nov 6 11:31 353.c
-rw-r--r-- 1 tch users 313 Nov 3 00:29 354.c
-rwxr-xr-x 1 tch users 5.8K Nov 6 13:21 consumer
-rw-r--r-- 1 tch users 717 Nov 6 13:43 consumer.c
-rwxr-xr-x 1 tch users 181 Nov 5 14:30 count.sh
-rw-r--r-- 1 tch users 112 Nov 6 13:49 count.txt
-rwxr-xr-x 1 tch users 5.9K Nov 6 13:20 producer
-rw-r--r-- 1 tch users 667 Nov 6 13:18 producer.c
-rwxr-xr-x 1 tch users 5.7K Nov 5 13:53 test_execl
-rw-r--r-- 1 tch users 845 Nov 5 13:51 test_execl.c
-rwxr-xr-x 1 tch users 5.5K Nov 5 13:13 test_fork
-rw-r--r-- 1 tch users 791 Nov 5 13:49 test_fork.c
-rw-r--r-- 1 tch users 463 Oct 31 09:16 test_fork_wait.c
-rwxr-xr-x 1 tch users 5.8K Nov 6 02:54 test_shm_A
-rw-r--r-- 1 tch users 1.3K Nov 6 02:54 test_shm_A.c
-rwxr-xr-x 1 tch users 5.7K Nov 6 02:54 test_shm_B
-rw-r--r-- 1 tch users 1.2K Nov 6 02:54 test_shm_B.c
-rwxr-xr-x 1 tch users 5.3K Nov 5 13:55 test_system
-rw-r--r-- 1 tch users 573 Oct 31 08:50 test_system.c
-rwxr-xr-x 1 tch users 6.2K Nov 6 13:55 time
-rw-r--r-- 1 tch users 823 Nov 6 13:54 time.c
Thời gian thực thi: 0.002398 giây.
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time abc
time: abc: No such file or directory
Thời gian thực thi: 0.000208 giây.
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time
time: Cách dùng: ./time <trình> [tham số ...]
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

- **Giải thích:** nếu không có tham số truyền vào thì in cách dùng rồi thoát. Tạo vùng nhớ chia sẻ có độ dài bằng độ dài của struct timeval (gồm 2 giá trị: tv_sec là thời gian dạng giây chẵn, tv_usec là phần lẻ dưới dạng micro giây). Sau đó dùng fork() tạo tiến trình con, trong tiến trình con lấy mốc thời gian đầu bằng hàm gettimeofday(). Sau đó cho thực thi lệnh được nhập vào. Ở tiến trình cha, chờ tới lúc tiến trình con kết thúc rồi lấy mốc thời gian thứ 2. Lấy mốc 2 trừ mốc 1 được thời gian chạy của lệnh, in ra thời gian tính được, hủy vùng nhớ chia sẻ rồi kết thúc.

+ Nếu mmap(), fork() hoặc execvp() gặp lỗi thì báo lỗi rồi thoát.

3.5.3. Viết một chương trình làm bốn công việc sau theo thứ tự: In ra dòng chữ: "Welcome to IT007, I am <your_Student_ID>!", thực thi file script count.sh với số lần đếm là 120, trước khi count.sh đếm đến 120, bấm CTRL+C để dừng tiến trình này, khi người dùng nhấn CTRL+C thì in ra dòng chữ: "count.sh has stoppped"

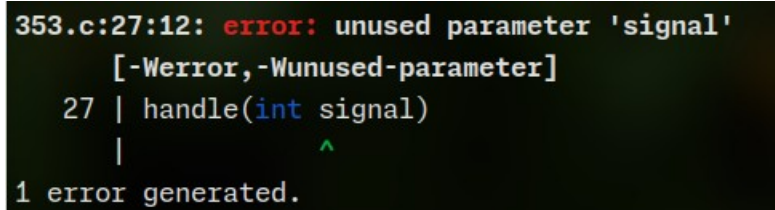
- **Phương pháp:** tạo tiến trình con. Chạy script ở tiến trình con, nhận tín hiệu ở tiến trình cha, khi có tín hiệu thì tiến trình cha in thông báo.



```
27 #include <sys/wait.h>
28 #include <err.h>
29 #include <signal.h>
30 #include <stdio.h>
31 #include <stdlib.h>
32 #include <unistd.h>
33
34 void handle(int);
35
36 int
37 main(void)
38 {
39     printf("Welcome to IT007, I am 24520346!\n");
40
41     switch (fork()) {
42     case -1:
43         err(1, "fork");
44     case 0:
45         execl("./count.sh", "./count.sh", "120", NULL);
46     default:
47         signal(SIGINT, handle);
48         wait(NULL);
49     }
50
51     exit(0);
52 }
53
54 void
55 handle(int __attribute__((unused)) signal)
56 {
57     printf("count.sh has stoppped.\n");
58 }
```

```
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc 353.c -o 353
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./353
Welcome to IT007, I am 24520346!
Implementing: ./count.sh
PPID of count.sh:
tch  24810 24809  0 13:49 pts/2    00:00:00 /bin/bash ./count.sh 120
tch  24812 24810  0 13:49 pts/2    00:00:00 grep count.sh
^Ccount.sh has stoppped.
tch:~/ATT/TK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

- **Giải thích:** đầu tiên, in dòng Welcome, sau đó tạo tiến trình con chạy count.sh. Ở tiến trình cha, thực hiện hàm handle khi nhận tín hiệu SIGINT (tín hiệu khi dùng Ctrl+C dừng tiến trình). Ở hàm handle, in thông báo. Do ta không dùng giá trị trả về của signal(), phải dùng attribute unused được để tránh báo lỗi từ compiler:



```
353.c:27:12: error: unused parameter 'signal'
      [-Werror, -Wunused-parameter]
27 | handle(int signal)
    |           ^
1 error generated.
```

Nếu fork() thất bại thì báo lỗi rồi thoát.

3.5.4. Viết chương trình mô phỏng bài toán Producer - Consumer như sau: Sử dụng kỹ thuật shared-memory để tạo một bounded-buffer có độ lớn là 10 bytes. Tiến trình cha đóng vai trò là Producer, tạo một số ngẫu nhiên trong khoảng [10, 20] và ghi dữ liệu vào buffer. Tiến trình con đóng vai trò là Consumer đọc dữ liệu từ buffer, in ra màn hình và tính tổng. Khi tổng lớn hơn 100 thì cả 2 dừng lại.

- **Phương pháp:** ở Producer, tạo vùng nhớ chia sẻ giữa 2 tiến trình, tiến hành lặp lại việc tạo 1 số ngẫu nhiên rồi đưa vào vùng nhớ. Ở Consumer, đọc vùng nhớ chia sẻ, in số đọc được và tổng tới khi tổng quá 100 thì dừng, gán lại vùng nhớ thành tín hiệu dừng. Khi đó, Producer dừng tạo số và cả 2 tiến trình kết thúc.

```
7 #include <sys/mman.h>
8 #include <sys/time.h>
9 #include <sys/wait.h>
10 #include <err.h>
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <unistd.h>
14
15 int
16 main(int argc, char *argv[])
17 {
18     struct timeval *first, second;
19
20     if (argc < 2)
21         errx(0, "Cách dùng: %s <ctrình> [tham số ...]", argv[0]);
22
23     first = mmap(0, sizeof(struct timeval), PROT_READ | PROT_WRITE,
24                 MAP_SHARED | MAP_ANON, -1, 0);
25     if (first == MAP_FAILED)
26         err(1, "mmap");
27
28     switch (fork()) {
29         case -1:
30             err(2, "fork");
31         case 0:
32             gettimeofday(first, NULL);
33             execvp(argv[1], &argv[1]);
34             err(3, "%s", argv[1]);
35         default:
36             wait(NULL);
37             gettimeofday(&second, NULL);
38             timersub(&second, first, first);
39             printf("Thời gian thực thi: %ld.%06ld giây.\n",
40                   first->tv_sec, first->tv_usec);
41     }
42
43     munmap(first, sizeof(struct timeval));
44     return 0;
45 }
```

```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc time.c -o time
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time ls -lh
total 106K
-rwxr-xr-x 1 tch users 5.6K Nov 6 13:49 353
-rw-r--r-- 1 tch users 491 Nov 6 11:31 353.c
-rw-r--r-- 1 tch users 313 Nov 3 00:29 354.c
-rwxr-xr-x 1 tch users 5.8K Nov 6 13:21 consumer
-rw-r--r-- 1 tch users 717 Nov 6 13:43 consumer.c
-rwxr-xr-x 1 tch users 181 Nov 5 14:30 count.sh
-rw-r--r-- 1 tch users 112 Nov 6 13:49 count.txt
-rwxr-xr-x 1 tch users 5.9K Nov 6 13:20 producer
-rw-r--r-- 1 tch users 657 Nov 6 13:18 producer.c
-rwxr-xr-x 1 tch users 5.7K Nov 5 13:53 test_exec1
-rw-r--r-- 1 tch users 845 Nov 5 13:51 test_exec1.c
-rwxr-xr-x 1 tch users 5.5K Nov 5 13:13 test_fork
-rw-r--r-- 1 tch users 791 Nov 5 13:49 test_fork.c
-rw-r--r-- 1 tch users 463 Oct 31 09:16 test_fork_wait.c
-rwxr-xr-x 1 tch users 5.8K Nov 6 02:54 test_shm_A
-rw-r--r-- 1 tch users 1.3K Nov 6 02:54 test_shm_A.c
-rwxr-xr-x 1 tch users 5.7K Nov 6 02:54 test_shm_B
-rw-r--r-- 1 tch users 1.2K Nov 6 02:54 test_shm_B.c
-rwxr-xr-x 1 tch users 5.3K Nov 5 13:55 test_system
-rw-r--r-- 1 tch users 573 Oct 31 08:50 test_system.c
-rwxr-xr-x 1 tch users 6.2K Nov 6 13:55 time
-rw-r--r-- 1 tch users 823 Nov 6 13:54 time.c
Thời gian thực thi: 0.002398 giây.
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time abc
time: abc: No such file or directory
Thời gian thực thi: 0.000208 giây.
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./time
time: Cách dùng: ./time <ctrình> [tham số ...]
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```



```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc producer.c -o producer
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./producer
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc consumer.c -o consumer
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./consumer
13, sum = 13
15, sum = 28
16, sum = 44
19, sum = 63
19, sum = 82
19, sum = 101
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

[0] 0: bash*

"tch@localPC vd" 14:09 06-Nov-25

- **Giải thích:** ở Producer, tạo vùng nhớ chia sẻ tên /SHM có độ dài 10 byte, với quyền 0600. Mã quyền tương tự như dùng với lệnh chmod. Ngầm định *ptr = -1 là điều kiện dừng lặp, bắt đầu vòng lặp ghi ngẫu nhiên một số từ 10 tới 20 vào *ptr rồi chờ trong 0.5 giây. Khi hết vòng lặp, Producer ngừng đọc/ghi vào vùng nhớ.

Ở Consumer, mở vùng nhớ có tên /SHM với quyền đọc/ghi. Nếu chưa tồn tại thì chờ, mỗi lần kiểm tra cách nhau 0.5 giây. Sau đó, khởi tạo sum = 0, khi sum $\leq$ 100 thì lấy giá trị *ptr về lưu tạm rồi cộng vào sum, sau đó in biến tạm đó và sum ra màn hình rồi chờ 0.5 giây. Khi sum > 100, vòng lặp dừng, gán *ptr = -1 để Producer ngừng lặp, thu hồi vùng nhớ chia sẻ.

Nếu shm_open(), ftruncate(), mmap(), munmap(), close() hoặc shm_unlink() gặp lỗi thì báo lỗi rồi thoát.

3.6. Viết chương trình tính toán chuỗi theo phỏng đoán Collatz

- **Phương pháp:** tạo vùng nhớ chia sẻ giữa tiến trình con và cha. Tiến trình con tính chuỗi Collatz lưu vào vùng nhớ chia sẻ, tiến trình cha đọc vùng nhớ rồi in chuỗi.

```
11 #include <unistd.h>
12
13 int
14 main(int argc, char *argv[])
15 {
16     long long *ptr, num;
17
18     if (argc < 2)
19     {
20         fprintf(stderr, "Cách dùng: %s <số nguyên dương>", argv[0]);
21         num = atoll(argv[1]);
22         if (num < 1)
23         {
24             fprintf(stderr, "Số bé hơn 1, thoát.", num);
25             ptr = mmap(0, sizeof(long long) * 4096, PROT_READ | PROT_WRITE,
26                 MAP_SHARED | MAP_ANON, -1, 0);
27             if (ptr == MAP_FAILED)
28                 fprintf(stderr, "mmap");
29
30             switch (fork()) {
31                 case -1:
32                     fprintf(stderr, "fork");
33                 case 0:
34                     ptr[0] = num;
35                     for (int i = 1; num > 1 && i < 4096; i++) {
36                         if (num % 2 == 0)
37                             num /= 2;
38                         else
39                             num = 3 * num + 1;
40                         ptr[i] = num;
41                     }
42                     exit(0);
43                 default:
44                     wait(NULL);
45                     for (int i = 0; ptr[i] >= 1 && i < 4096; i++)
46                         printf("%lld", ptr[i], (ptr[i] == 1) ?
47                             "\n" : ", ");
48             }
49             munmap(ptr, sizeof(long long) * 4096);
50             return 0;
51 }
52 }
```

```
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ cc collatz.c -o collatz
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./collatz 35
35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./collatz 1
1
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./collatz 0
collatz: 0 bé hơn 1, thoát.
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$ ./collatz
collatz: Cách dùng: ./collatz <số nguyên dương>
tch:~/ATTT/HK3_25-26/IT007.Q17/TH/Lab3/vd
$
```

- **Giải thích:** chương trình đọc số lấy được, chuyển từ dạng ký tự sang số. Sau đó, tạo vùng nhớ chia sẻ và tiến trình con. Ở tiến trình con, coi vùng nhớ như một mảng, tính phần tử rồi lưu vào mảng, tới khi phần tử bằng 1 thì dừng. Ở tiến trình cha, đợi tiến trình con hoàn thành rồi in chuỗi số từ vùng nhớ, thu hồi vùng nhớ rồi thoát.